

DevOps Lab Assessment

Name: Bokhit mahamat tom

PRN: 23070122249

CSE-B2

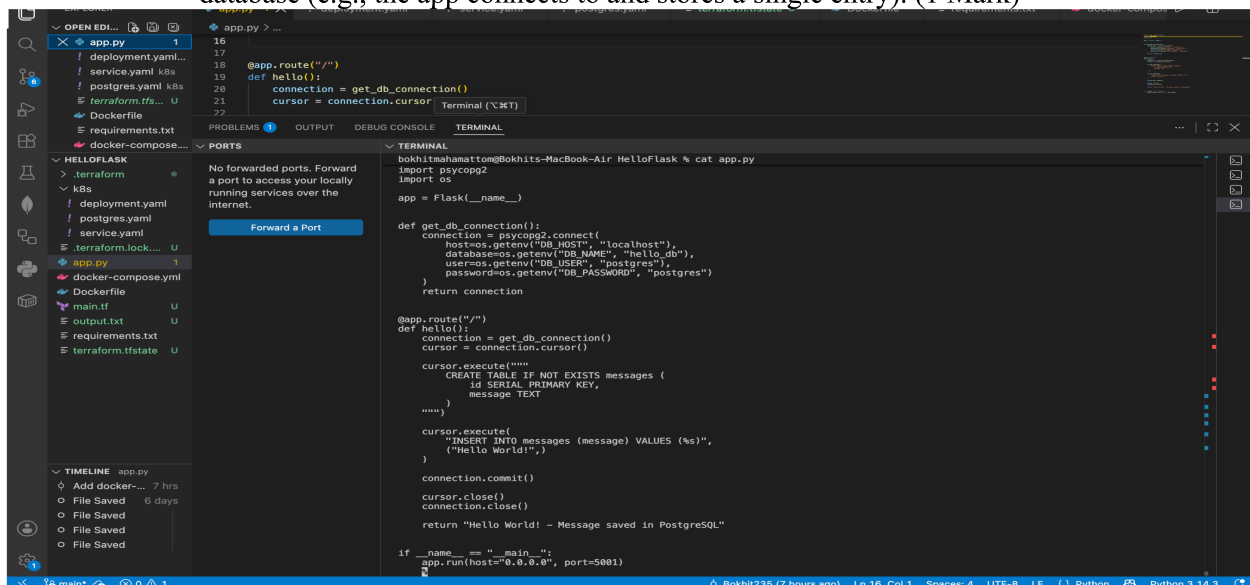
2. TW2 - Continuous Evaluation: Advanced DevOps Practices (10 Marks)

Instructions: Build upon your previous work. Document your commands, configuration files (e.g., docker-compose.yml, Kubernetes YAMLs), and outputs.

Scenario: You need to enhance your "Hello World" application to include a database and deploy it using orchestration tools.

1. Docker Compose for Multi-Container App (3 Marks)

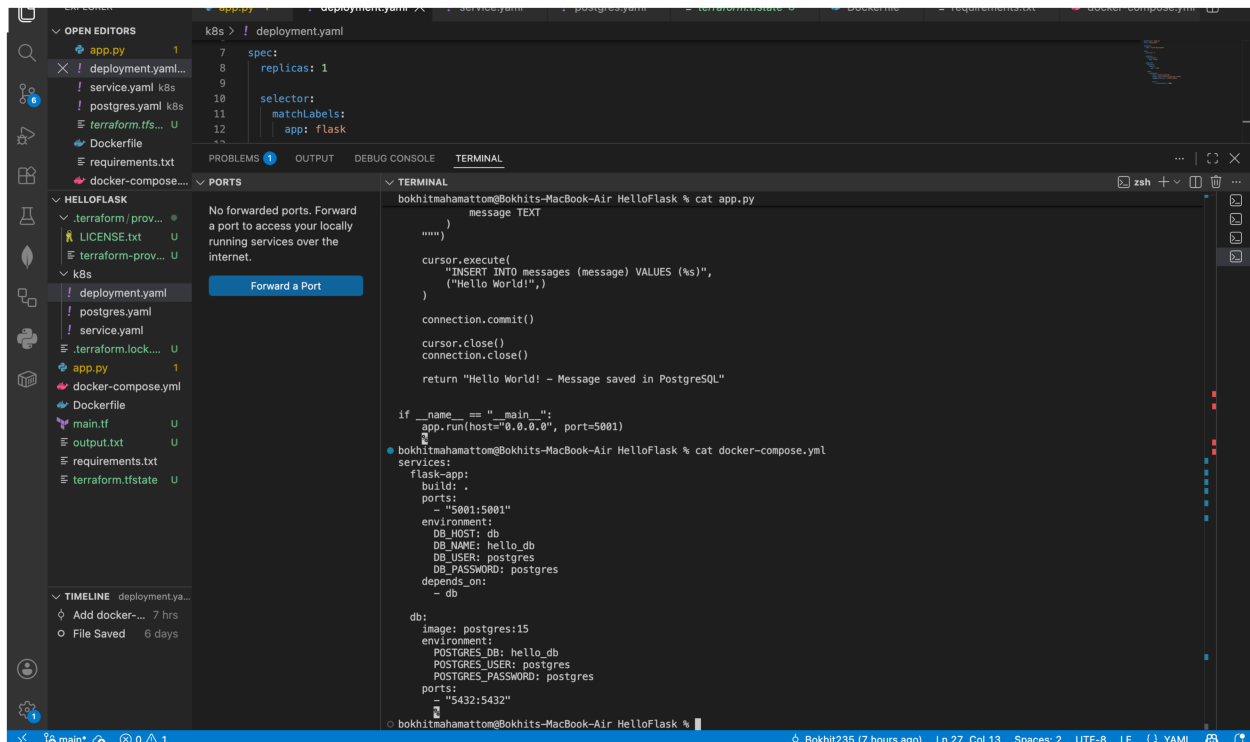
- Task 1.1: Modify your "Hello World" Flask application to include a simple PostgreSQL database (e.g., the app connects to and stores a single entry). (1 Mark)



The screenshot shows a code editor with a file explorer on the left. The file explorer lists files for 'HELLOFLASK', including 'deployment.yaml', 'k8s', 'postgres.yaml', 'terraform.tfstate', 'Dockerfile', 'requirements.txt', and 'docker-compose.yml'. The main editor displays the 'app.py' file, which contains the following code:

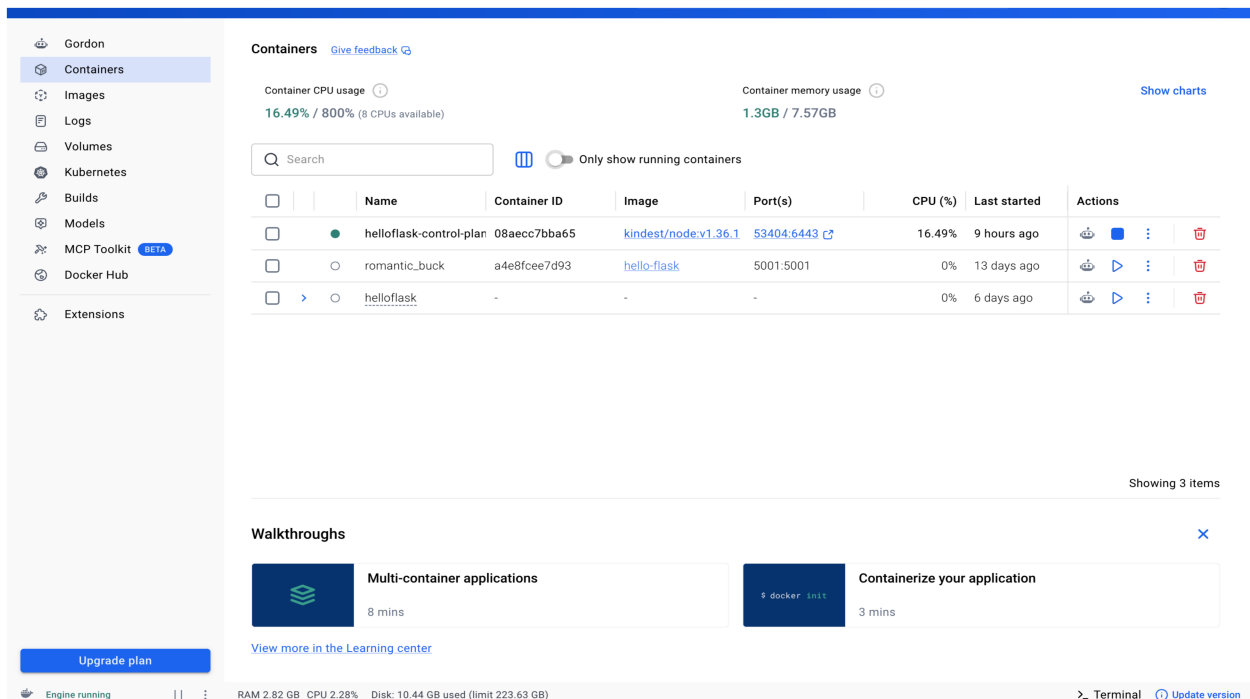
```
16
17
18 @app.route("/")
19 def hello():
20     connection = get_db_connection()
21     cursor = connection.cursor()
22
23     def get_db_connection():
24         connection = psycopg2.connect(
25             host=os.getenv("DB_HOST", "localhost"),
26             database=os.getenv("DB_NAME", "hello_db"),
27             user=os.getenv("DB_USER", "postgres"),
28             password=os.getenv("DB_PASSWORD", "postgres")
29         )
30         return connection
31
32     @app.route("/")
33     def hello():
34         connection = get_db_connection()
35         cursor = connection.cursor()
36
37         cursor.execute("""
38             CREATE TABLE IF NOT EXISTS messages (
39                 id SERIAL PRIMARY KEY,
40                 message TEXT
41             )
42         """)
43
44         cursor.execute(
45             "INSERT INTO messages (message) VALUES (%s)",
46             ("Hello World!",)
47         )
48
49         connection.commit()
50         cursor.close()
51         connection.close()
52
53         return "Hello World! - Message saved in PostgreSQL"
54
55 if __name__ == "__main__":
56     app.run(host="0.0.0.0", port=5001)
```

- Task 1.2: Create a docker-compose.yml file that defines two services: your Flask application and the PostgreSQL database. Configure them to communicate. Bring up the application using docker-compose up and verify both services are running and communicating. Provide the docker-compose.yml and output. (2 Marks)

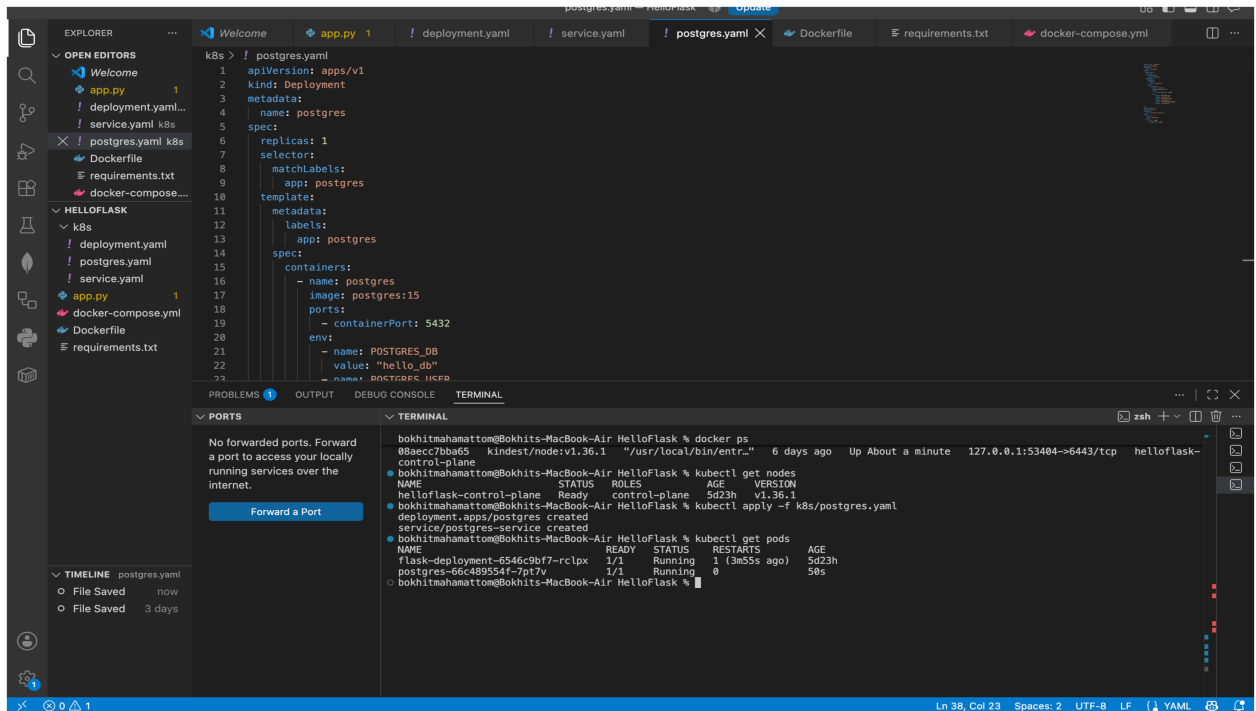


2. Kubernetes Deployment & Service (4 Marks)

- **Task 2.1:** Ensure your Docker image from TW1 (or an updated one) is accessible (e.g., pushed to a public Docker Hub repository, or Minikube's Docker daemon is used).

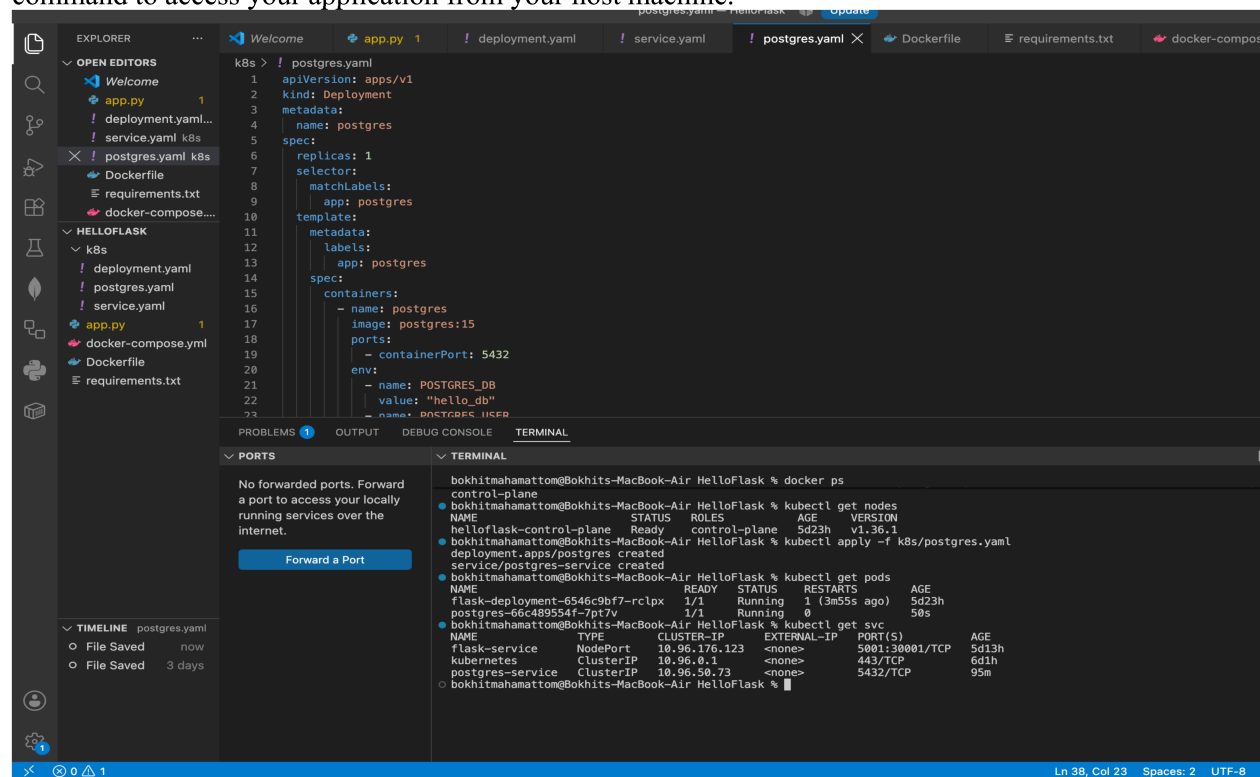


- **Task 2.2:** Create a Kubernetes Deployment YAML for your Flask application. Deploy it to your local Kubernetes cluster (Minikube/Kind). Verify the Pods are running. (1.5 Marks)



3.

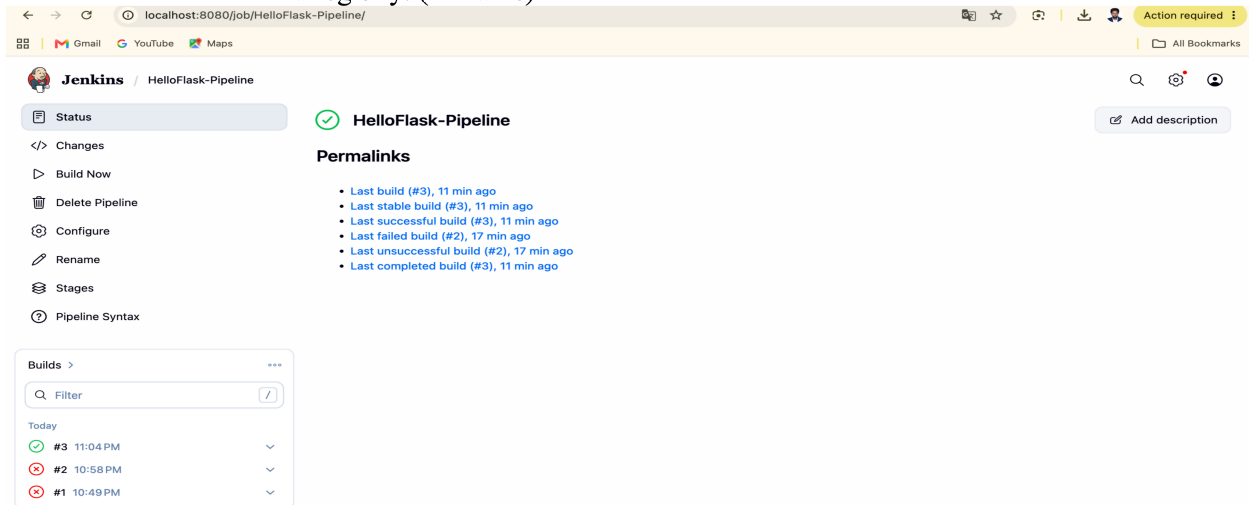
- **Task 2.3:** Create a Kubernetes Service YAML (type NodePort or LoadBalancer if supported) to expose your Flask application. Apply the manifest and provide the command to access your application from your host machine.



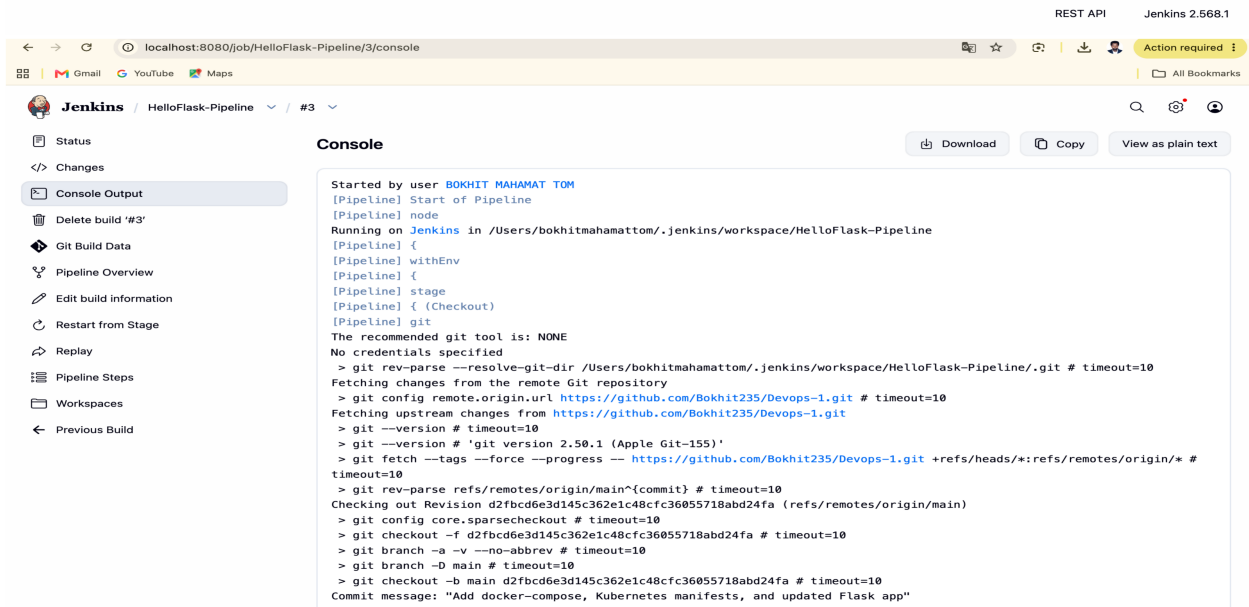
4. Integrated CI/CD with Jenkins & Basic Infrastructure as Code (IaC) (3 Marks)

- **Task 3.1:** Create a Jenkins Declarative Pipeline that performs the following steps:
 - Checks out your Git repository.
 - Builds the Docker image for your Flask application (from Task 1.1).

- (Optional but Recommended for higher scores): Pushes the built Docker image to a registry. (2 Marks)



The screenshot shows the Jenkins web interface for the 'HelloFlask-Pipeline' job. The left sidebar contains navigation links: Status (selected), Changes, Build Now, Delete Pipeline, Configure, Rename, Stages, and Pipeline Syntax. The main content area displays the job name 'HelloFlask-Pipeline' with a green checkmark icon. Below it, there is a 'Permalinks' section listing various build links such as 'Last build (#3), 11 min ago', 'Last stable build (#3), 11 min ago', 'Last successful build (#3), 11 min ago', 'Last failed build (#2), 17 min ago', 'Last unsuccessful build (#2), 17 min ago', and 'Last completed build (#3), 11 min ago'. On the right, there is an 'Add description' button. At the bottom left, a 'Builds' dropdown menu is open, showing a list of recent builds: #3 (11:04 PM), #2 (10:58 PM), and #1 (10:49 PM), each with a status icon (green for success, red for failure).



The screenshot shows the Jenkins web interface for the 'HelloFlask-Pipeline' job, specifically the console output for build #3. The left sidebar contains navigation links: Status, Changes, Console Output (selected), Delete build '#3', Git Build Data, Pipeline Overview, Edit build information, Restart from Stage, Replay, Pipeline Steps, Workspaces, and Previous Build. The main content area displays the console output, which includes the following text:

```
Started by user BOKHIT MAHAMAT TOM
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /Users/bokhitmahamat/.jenkins/workspace/HelloFlask-Pipeline
[Pipeline] {
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Checkout)
[Pipeline] git
The recommended git tool is: NONE
No credentials specified
> git rev-parse --resolve-git-dir /Users/bokhitmahamat/.jenkins/workspace/HelloFlask-Pipeline/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/Bokhit235/Devops-1.git # timeout=10
Fetching upstream changes from https://github.com/Bokhit235/Devops-1.git
> git --version # timeout=10
> git --version # 'git version 2.50.1 (Apple Git-155)'
> git fetch --tags --force --progress -- https://github.com/Bokhit235/Devops-1.git +refs/heads/*:refs/remotes/origin/* #
timeout=10
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision d2fbc6e3d145c362e1c48cfc36055718abd24fa (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f d2fbc6e3d145c362e1c48cfc36055718abd24fa # timeout=10
> git branch -a -v --no-abbrev # timeout=10
> git branch -D main # timeout=10
> git checkout -b main d2fbc6e3d145c362e1c48cfc36055718abd24fa # timeout=10
Commit message: "Add docker-compose, Kubernetes manifests, and updated Flask app"
..
```


- **Task 3.2:** Install Terraform. Write a simple Terraform configuration (main.tf) to create a local file (e.g., output.txt) with some content. Initialize, plan, and apply the configuration. Show the output of terraform plan and verify the file creation. (1 Mark)

```

# file_permission = "0777"
# filename = "~/.output.txt"
# id = (known after apply)
}

Plan: 1 to add, 0 to change, 0 to destroy.

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.
bokhithamahamattom192 HelloFlask % terraform apply

Terraform used the selected providers to generate the following execution plan.
Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# local_file.output will be created
+ resource "local_file" "output" {
  + content = "Hello from Terraform! This file was created automatically."
    - content_base64sha256 = (known after apply)
    - content_base64sha512 = (known after apply)
    - content_md5 = (known after apply)
    - content_sha1 = (known after apply)
    - content_sha256 = (known after apply)
    - content_sha512 = (known after apply)
    - directory_permissions = "0777"
    - file_permission = "0777"
    - filename = "~/.output.txt"
    - id = (known after apply)
}

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

local_file.output: Creating...
local_file.output: Creation complete after 0s [id=7842f1372b3b23dd5867143f62aea0609f54696d]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
bokhithamahamattom192 HelloFlask % ls -la
total 64
drwxr-xr-x 13 bokhithamahamattom staff 416 16 Aug 16:49 .
drwxr-xr-x 86 bokhithamahamattom staff 2752 16 Aug 16:24 ..
drwxr-xr-x 13 bokhithamahamattom staff 416 16 Aug 16:27 git
drwxr-xr-x 3 bokhithamahamattom staff 96 16 Aug 16:46 terraform
-rw-r--r-- 1 bokhithamahamattom staff 1278 16 Aug 16:46 terraform.lock.hcl
-rw-r--r-- 1 bokhithamahamattom staff 917 18 Aug 13:17 app.py
-rw-r--r-- 1 bokhithamahamattom staff 381 18 Aug 13:29 docker-compose.yml
-rw-r--r-- 1 bokhithamahamattom staff 167 18 Aug 13:26 Dockerfile
drwxr-xr-x 5 bokhithamahamattom staff 160 13 Aug 14:08 k8s
-rw-r--r-- 1 bokhithamahamattom staff 160 16 Aug 16:46 main.tf
-rwxr-xr-x 1 bokhithamahamattom staff 58 16 Aug 16:49 output.txt
-rw-r--r-- 2 bokhithamahamattom staff 21 18 Aug 13:26 requirements.txt
-rw-r--r-- 1 bokhithamahamattom staff 1673 16 Aug 16:49 terraform.tfstate
bokhithamahamattom192 HelloFlask % cat output.txt
bokhithamahamattom192 HelloFlask %

```

EXPLORER

OPEN EDITORS

app.py 1

deployment.yaml...

service.yaml k8s

postgres.yaml k8s

terraform.tfstate U

Dockerfile

requirements.txt

docker-compose.yml...

HELL...

.terraform

k8s

deployment.yaml

postgres.yaml

service.yaml

.terraform.lock.... U

app.py 1

docker-compose.yml

Dockerfile

main.tf U

output.txt U

requirements.txt

terraform.tfstate U

Local History will track recent changes as you save them unless the file has been excluded or is too large.

PROBLEMS 1

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

No forwarded ports. Forward a port to access your locally running services over the internet.

Forward a Port

TERMINAL

bokhitmahamattom@Bokhits-MacBook-Air HelloFlask % docker ps

bokhitmahamattom@Bokhits-MacBook-Air HelloFlask % kubectl get nodes

NAME STATUS ROLES AGE VERSION

helloflask-control-plane Ready control-plane 5d23h v1.36.1

bokhitmahamattom@Bokhits-MacBook-Air HelloFlask % kubectl apply -f k8s/postgres.yaml

deployment.apps/postgres created

service/postgres-service created

bokhitmahamattom@Bokhits-MacBook-Air HelloFlask % kubectl get pods

NAME READY STATUS RESTARTS AGE

flask-deployment-6546c9bf7-rc1px 1/1 Running 1 (3m55s ago) 5d23h

postgres-66c489554f-7pt7v 1/1 Running 0 50s

bokhitmahamattom@Bokhits-MacBook-Air HelloFlask % kubectl get svc

NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE

flask-service NodePort 10.96.176.123 <none> 5081:30801/TCP 5d13h

kubernetes ClusterIP 10.96.0.1 <none> 443/TCP 6d1h

postgres-service ClusterIP 10.96.50.73 <none> 5432/TCP 95m

bokhitmahamattom@Bokhits-MacBook-Air HelloFlask %

Ln 1, Col 1 Spaces: 2 UTF-8 LF Plain Text